Fall 2025

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Lecture 5

- Recommended Reading:
  - Chapter 4
  - Note: we will not be covering the Master Theorem version in this chapter or Strassen's algorithm for fast matrix multiplication

- Preliminaries
- Divide and Conquer
  - Master Method
  - Maximum-Sum Subarray

Geometric Series

Expression for finite geometric series

$ar^0 = a$

$S = a + ar^1 + ar^2 + ar^3 + \ldots + ar^k$ finite geometric series

$$= \sum_{i=0}^{k} ar^i$$

$$= \begin{cases} a \cdot \dfrac{r^{k+1} - 1}{r - 1} & \text{if } r \neq 1 \\ a(k+1) & \text{if } r = 1 \end{cases}$$

- if r is constant
  - $= \theta(ak)$ if $r = 1$
  - $= \theta(ar^k)$ if $r > 1$ (also $= \theta(ar^{k+1})$) theta of last term
  - $= \theta(a)$ if $r < 1$ ($= \theta(1)$ if a constant) theta of first term

$r = 1$: $a + a(1) + a(1)^2 + a(1)^3 + \ldots$

a $k+1$ times

$rS - S = ar^{k+1} - a$

$S(r-1) = ar^{k+1} - a$

$S = a \cdot \dfrac{r^{k+1} - 1}{r - 1}$

$r = \frac{1}{2}$ $a + \frac{1}{2}a + \frac{1}{4}a + \cdots \left(\frac{1}{2}\right)^k a$

Examples of finite geometric series

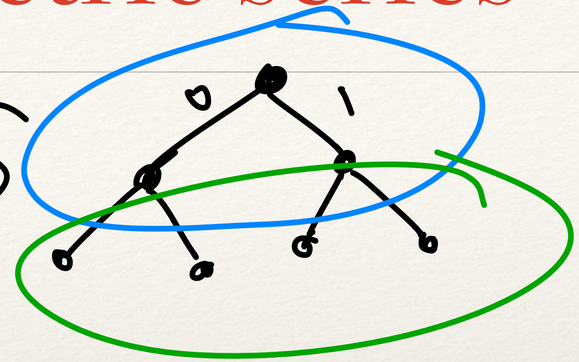
$a = 1, r = 2$

$$\sum_{i=0}^{k} 2^i = 1 + 2 + 4 + \ldots + 2^k$$

$= \text{number of nodes of binary tree of depth } k$

$$= \frac{2^{k+1} - 1}{2 - 1} = 2^{k+1} - 1 = 2^k + 2^k - 1$$

Binary tree of depth k has 2^k leaves and $2^k - 1$ internal nodes



$d = 2$

$2^2 = 4$ leaves

3 interval

$a = 1$

$r = \frac{1}{2}$

$$\sum_{i=1}^{k} \frac{1}{2}^i = 1 + \frac{1}{2} + \frac{1}{4} + \ldots + \frac{1}{2^k} = \frac{\frac{1}{2}^{k+1} - 1}{\frac{1}{2} - 1} = 2 \cdot (1 - \frac{1}{2}^{k+1})$$

< 1

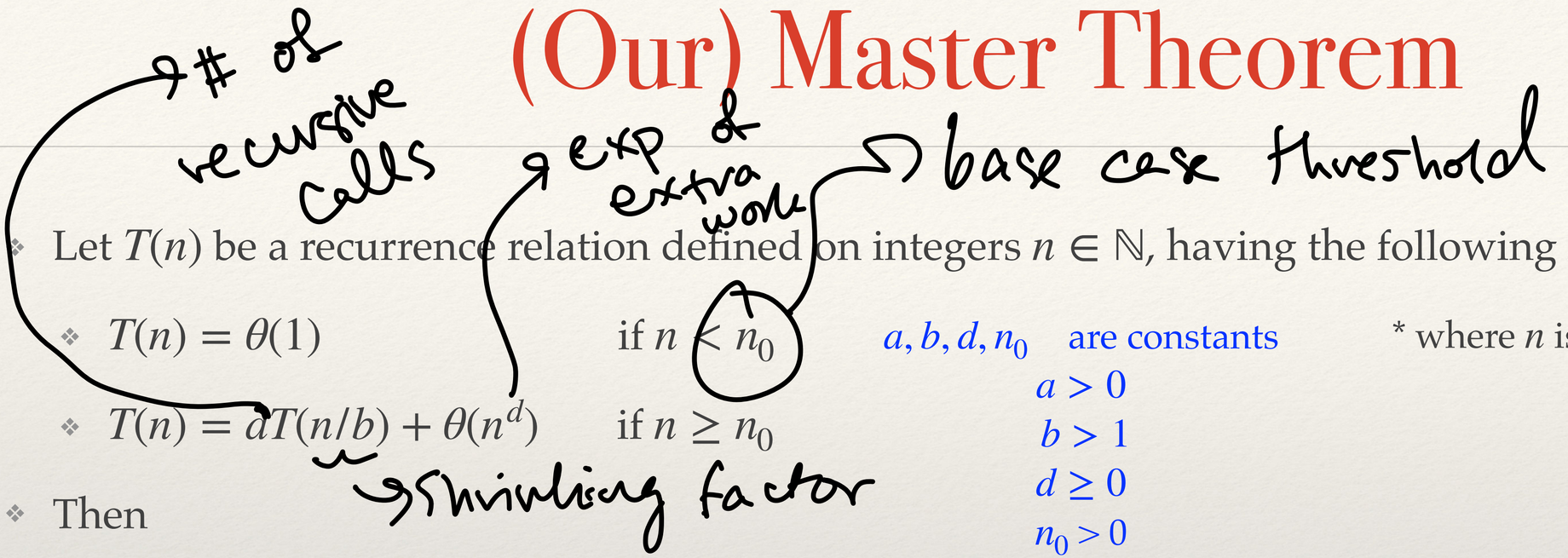
This is less than 2. Not surprising intuitively, why?

< 1

< 2

Master Method

(Our) Master Theorem



- Let $T(n)$ be a recurrence relation defined on integers $n \in \mathbb{N}$, having the following form:
  - $T(n) = \theta(1)$ if $n < n_0$
  - $T(n) = aT(n/b) + \theta(n^d)$ if $n \geq n_0$

a, b, d, n_0 are constants

$a > 0$
$b > 1$
$d \geq 0$
$n_0 > 0$

* where n is a power of b

- Then
  - $T(n) = \theta(n^d \log n)$ if $d = \log_b a$
  - $T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$
  - $T(n) = \theta(n^d)$ if $d > \log_b a$

Show this first

Here $aT(n/b)$ means $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ for some $a', a'' > 0$ where $a' + a'' = a$

Textbook has even more general version of this theorem

- Before continuing with proof of Master Theorem, let’s apply it!

$\frac{n}{b^i} = 1 \Rightarrow n = b^i \quad \log_b n = \log_b b^i \Rightarrow \log_b n = i$

Applying the Master Theorem

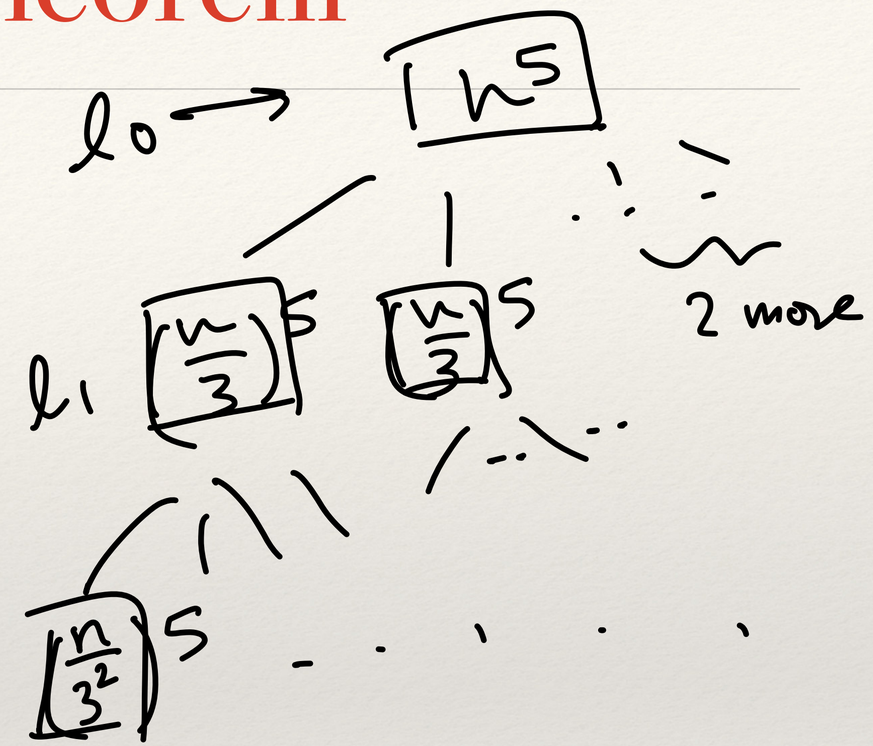
$T(n) = aT\left(\frac{n}{b}\right) + \theta(n^d)$

$a = 4$

$b = 3$

$n_0 = 2$

$d = 5$



$T(n) = \theta(n^d \log n)$ if $d = \log_b a$

$T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$

$T(n) = \theta(n^d)$ if $d > \log_b a$

- $T(n) = 4T(n/3) + 7n^5 \quad n > 2$
  $T(n) = 5$ when $n \leq 2$

By Master Theorem, $T(n) = \theta($ n^5 $)$

$d = 5$

$\log_b a = \log_3 4 = 1.3 < 5 = 5$

$\therefore \log_b n = n^{\log_b a}$

$a = 2$

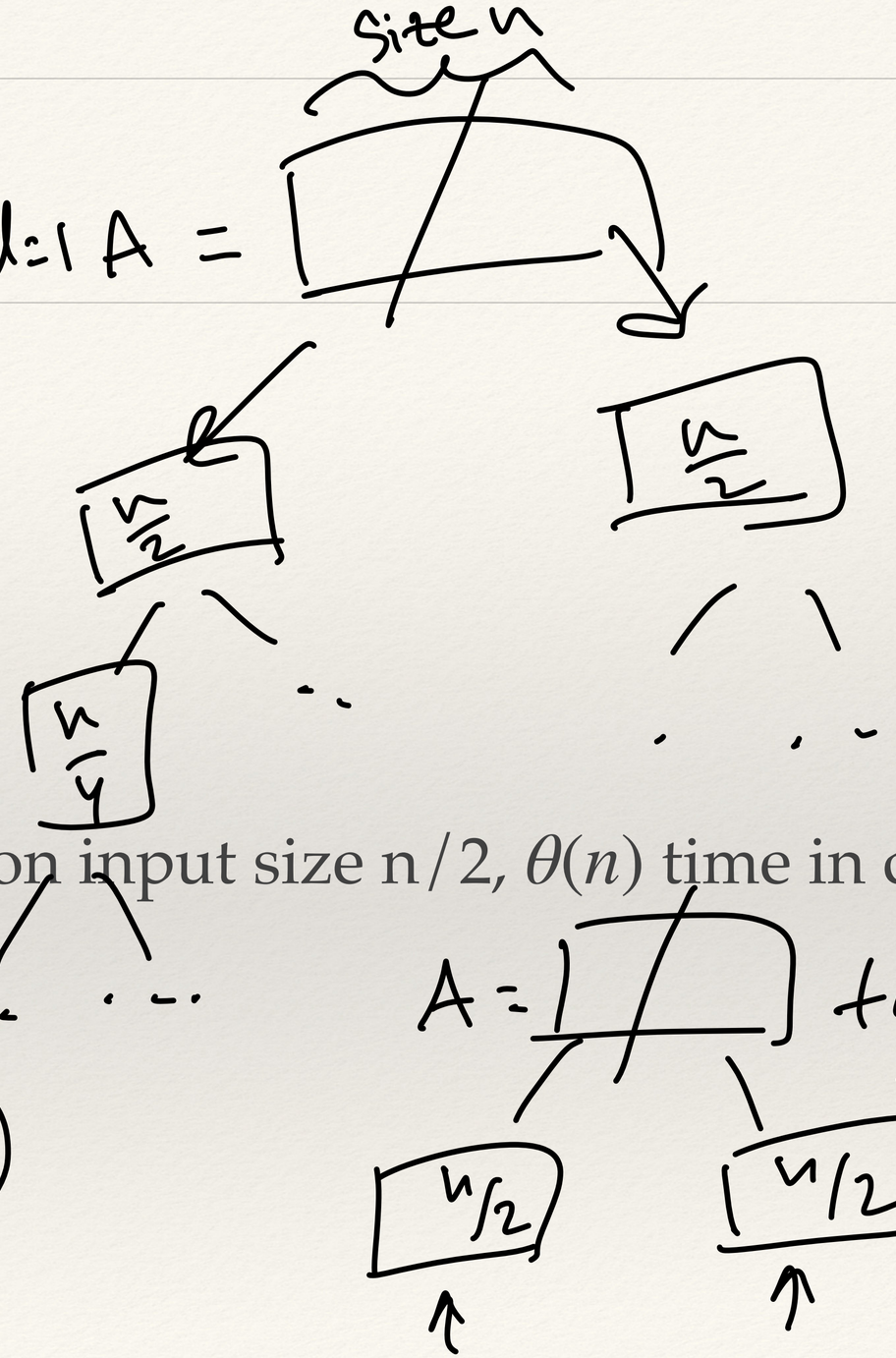
$b = 2$

$d = 1$

$\log_b a = \log_2 2 = 1 = d$

$\Theta(n^d \log n)$

$\Theta(n \log n)$



- Recurrence for Mergesort
  - $T(n) = 2T(n/2) + \theta(n)$

    Each call: makes 2 recursive calls, each on input size $n/2$, $\theta(n)$ time in call
- Recurrence for Binary Search?

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

$a = 1$

- Recurrence for Mergesort
  - $T(n) = 2T(n/2) + \theta(n)$
    Each call: makes 2 recursive calls, each on input size $n/2$, $\theta(n)$ time in call
- Recurrence for Binary Search?
  - $T(n) = T(n/2) + \theta(1)$ $= n^d = n^0 = 1$
  - a=1, b=2, d=0, so $\log_b a = \log_2 1 = 0 = d$
    Master Theorem says $T(n) = \theta(n^d \log n) = \theta(\log n)$

Can't always apply (our) Master Theorem

Master theorem only applies to recurrences of form $T(n) = aT(n/b) + \theta(n^d)$

Cannot apply Master Theorem to these!

- $T(n) = 3T(n/4) + \theta(2^n) \quad n > 4$
  $T(n) = 1 \quad n \leq 4$

  $2^n \neq n^d \quad \exists d$

- $T(n) = T(n/3) + T(2n/3) + \theta(n^4) \quad n > 1$
  $T(n) = 1 \quad n = 1$

  $b = 3 \quad b = \frac{3}{2}$

- $T(n) = T(\sqrt{n}) + 6n \quad n > 1$
  $T(n) = 1 \quad n = 1$

  $\frac{n}{b} \neq \sqrt{n} \quad \exists b$

(Our) Master Theorem

- Let $T(n)$ be a recurrence relation defined on $n \in \mathbb{N}$ of the following form:
  - $T(n) = aT(n/b) + \theta(n^d)$ if $n \geq n_0$
  - $T(n) = \theta(1)$ if $n < n_0$

a, b, d, n_0 are constants

$a > 0$
$b > 1$
$d \geq 0$
$n_0 > 0$

- Then
  - $T(n) = \theta(n^d \log n)$ if $d = \log_b a$
  - $T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$
  - $T(n) = \theta(n^d)$ if $d > \log_b a$

(Let's prove this now)

Solve recurrence for $d = \log_b a$

Solve simpler recurrence: Remove the thetas in the recurrence

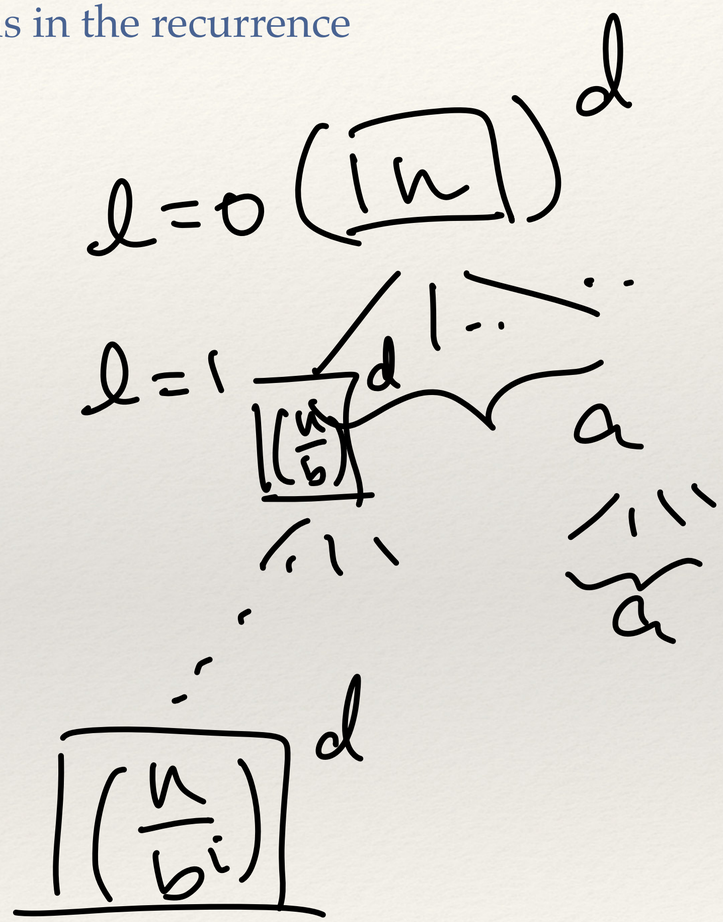
set $n_0 = 1$

Assume n a power of b

- $T(n) = aT(n/b) + n^d$ if $n > 1$
- $T(1) = 1$

$d = \log_b a$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(\frac{n}{b})^d$	$a(\frac{n}{b})^d$
2	n/b^2	a^2		
⋮			⋮	
i	n/b^i	a^i	$(\frac{n}{b^i})^d$	$a^i(\frac{n}{b^i})^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n}$



Solve recurrence for $d = \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$ $\qquad d = \log_b a$
- $T(1) = 1$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n}$

Time excluding base case: $\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i$

Solve recurrence for $d = \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$ $\qquad d = \log_b a$
- $T(1) = 1$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n} = n^{\log_b a}$

Total time:

$$\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i + n^{\log_b a}$$

Solve recurrence for $d = \log_b a$

- $T(n) = aT(n/b) + n^d \qquad \text{if } n > 1$
- $T(1) = 1$

$d = \log_b a$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n} = n^{\log_b a}$

$= n^d \left(\frac{a}{b^d}\right)^i$

$= n^d \left(\frac{a}{b^{\log_b a}}\right)^i$

$= n^d \left(\frac{a}{a}\right)^i$

$= n^d$

Total time:

$$\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i + n^{\log_b a} = \sum_{i=0}^{\log_b n - 1} n^d (a/a)^i + n^{\log_b a} = n^d \log_b n + n^d$$

why?

Solve recurrence for $d = \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$ $\qquad d = \log_b a$
- $T(1) = 1$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n} = n^{\log_b a}$

Total time:

$$\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i + n^{\log_b a} = \sum_{i=0}^{\log_b n - 1} n^d (a/a)^i + n^{\log_b a} = n^d \log_b n + n^d$$

using $d = \log_b a$

Solve recurrence for $d = \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$ $\quad d = \log_b a$
- $T(1) = 1$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n}$

$n^d \log_b n + n^d$

$\Theta(n^d \log_b n)$

$\Theta(n^d \log n)$

$T(n) = \theta(n^d \log n)$ when $d = \log_b a$

(Our) Master Theorem

- Let $T(n)$ be a recurrence relation defined on $n \in \mathbb{N}$ of the following form:
  - $T(n) = aT(n/b) + \theta(n^d)$ if $n \geq n_0$
  - $T(n) = \theta(1)$ if $n < n_0$

a, b, d, n_0 are constants
a, b are integers
$a > 0$
$b > 1$
$d \geq 0$
$n_0 > 0$

- Then
  - $T(n) = \theta(n^d \log n)$ if $d = \log_b a$
  - $T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$ ← Show this now
  - $T(n) = \theta(n^d)$ if $d > \log_b a$

Solve recurrence for $d < \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$ $\quad d < \log_b a$
- $T(1) = 1$

Solve simpler recurrence: Remove the thetas in the recurrence
set $n_0 = 1$
Assume n a power of b

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n}$

Total time

$$\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i + n^{\log_b a} = \theta(n^d (a/b^d)^{\log_b n}) + n^{\log_b a}$$

Simplify!

because $d < \log_b a \Rightarrow (a/b^d) > 1$ (sum of geometric series with $r > 1$)

$$\sum_{i=0}^{k} ar^k \qquad r = \left(\frac{a}{b^d}\right) > 1 \qquad \Theta(ar^k)$$

$$a^{\log_b n} = n^{\log_b a}$$

$$\left(\frac{a}{b^d}\right)^{\log_b n}$$

$$\frac{a^{\log_b n}}{(b^d)^{\log_b n}} = \frac{a^{\log_b n}}{n^d}$$

$$\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i + n^{\log_b a} = \theta(n^d (a/b^d)^{\log_b n}) + n^{\log_b a} \qquad \text{because } d < \log_b a \Rightarrow (a/b^d) > 1$$

$$= \theta(n^d (a^{\log_b n}/n^d)) + n^{\log_b a} \qquad \text{Why?}$$

$$= \theta(n^{\log_b a}) + n^{\log_b a}$$

$$= \theta(n^{\log_b a})$$

$$d < \log_b a$$

Solve recurrence for $d < \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$ $\quad d < \log_b a$
- $T(1) = 1$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n} = n^{\log_b a}$

$$T(n) = \theta(n^{\log_b a}) \qquad \text{when } d < \log_b a$$



(Our) Master Theorem

- Let $T(n)$ be a recurrence relation defined on $n \in \mathbb{N}$ of the following form:
  - $T(n) = aT(n/b) + \theta(n^d)$ if $n \geq n_0$
  - $T(n) = \theta(1)$ if $n < n_0$

  a, b, d, n_0 are constants

  $a > 0$

  $b > 1$

  $d \geq 0$

  $n_0 > 0$
- Then
  - $T(n) = \theta(n^d \log n)$ if $d = \log_b a$
  - $T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$
  - $T(n) = \theta(n^d)$ if $d > \log_b a$

Show this now

Analysis when $d > \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$
- $T(n) = 1$ if $n = 1$ where $d > log_b a$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n} = n^{\log_b a}$

Total time:

$$\sum_{i=0}^{\log_b n - 1} n^d (a/b^d)^i + n^{\log_b a} = \theta(n^d) + n^{\log_b a} = \theta(n^d)$$

because $d > \log_b a \Rightarrow (a/b^d) < 1$

Analysis when $d > \log_b a$

- $T(n) = aT(n/b) + n^d$ if $n > 1$
- $T(n) = 1$ if $n = 1$ where $d > log_b a$

Level #	Input size	#nodes	Time per node	Total Time
0	n	1	n^d	n^d
1	n/b	a	$(n/b)^d$	an^d/b^d
2	n/b^2	a^2	$(n/b^2)^d$	$a^2n^d/(b^2)^d$
⋮				
i	n/b^i	a^i	$(n/b^i)^d$	$a^i(n/b^i)^d = (a/b^d)^i n^d$
⋮				
$\log_b n - 1$				
$\log_b n$	1	$a^{\log_b n}$	1	$a^{\log_b n} = n^{\log_b a}$

$T(n) = \theta(n^d)$ when $d > \log_b$ Done with proof of Master Theorem!

(Our) Master Theorem

- Let $T(n)$ be a recurrence relation defined on integers $n \in \mathbb{N}$, having the following form:
  - $T(n) = \theta(1)$ if $n < n_0$
  - $T(n) = aT(n/b) + \theta(n^d)$ if $n \geq n_0$

  a, b, d, n_0 are constants
  $a > 0$
  $b > 1$
  $d \geq 0$
  $n_0 > 0$
- Then
  - $T(n) = \theta(n^d \log n)$ if $d = \log_b a$
  - $T(n) = \theta(n^{\log_b a})$ if $d < \log_b a$
  - $T(n) = \theta(n^d)$ if $d > \log_b a$

Here $aT(n/b)$ means $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ for some $a', a'' > 0$ where $a' + a'' = a$

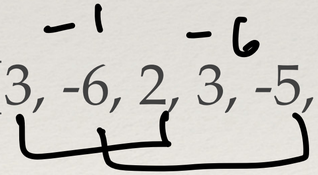
Textbook has even more general version of this theorem



Maximum Sum Subarray

Maximum Sum Subarray

- Maximum Sum Subarray Problem
  - Given array A of integers, find maximum value of sum of items in any subarray A[i…j] of A .
    - Note: Array A is not sorted, and the integers are not necessarily positive.
      - e.g., [3, -6, 2, 3, -5, 1, 3, -7]



[ ] = 0

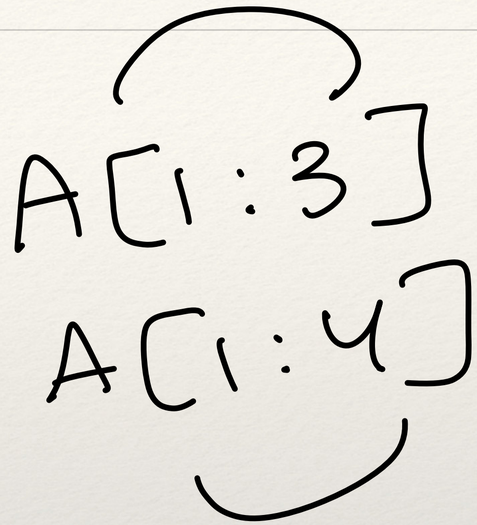
    - The empty subarray is also considered to be a subarray. Sum of its elements is 0.

[ ] = 0

Maximum Sum Subarray

- Approach #1: Brute force (iterative).
  For each subarray, sum up the items in it.
  (Sum up the items in each subarray from scratch, every time.)

- Pseudocode? Runtime in big-Oh notation?



$O(n^3)$

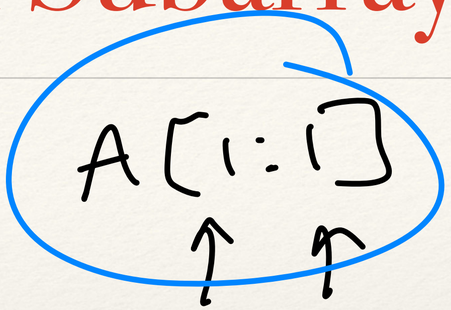
Maximum Sum Subarray

- Approach #1: Brute force (iterative). For each subarray, sum up the items in it.
  Sum up the items in each subarray from scratch, every time.

- Pseudocode:

```
maxsum = 0
for i=1 to n
    for j=i to n
        sum = 0
        for k=i to j  \\ loop to sum elements in A[i:j]
            sum += A[k]
        if sum > maxsum
            maxsum = sum

return sum
```



A[1:2] . . [1:n]

A[2:2] . . [2:n]

[3:3]...

$O(n) \cdot O(n) \cdot O(n) = O(n^3)$

Maximum Sum Subarray

$n = 4$

$A[1:1]$ $A[1:2]$ $A[1:3]$ $A[1:4]$

$A[2:2]$ $[2:4]$

$A[3:3]$

$A[4:4]$

- Approach #1: Brute force (iterative). For each subarray, sum up the items in it.
  Sum up the items in each subarray from scratch, every time.
  Runtime?

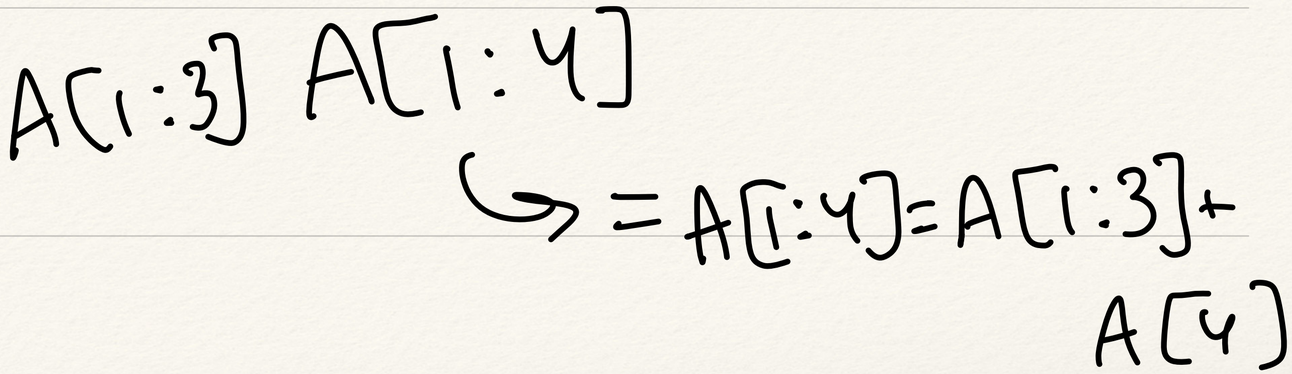
How many subarrays total?

n beginning in position 1
$n - 1$ beginning in position 2
$n - 2$ beginning in position 3
….
1 subarray beginning in position n

$\frac{n^2+n}{2} = \Theta(n^2)$

Total # subarrays = $1 + 2 + 3 + \ldots + n = n(n+1)/2 = \theta(n^2)$
How many elements in each subarray? At most? Answer: n
Runtime is $O(n^3)$
Can show this is a tight bound

$\Theta(n^3)$

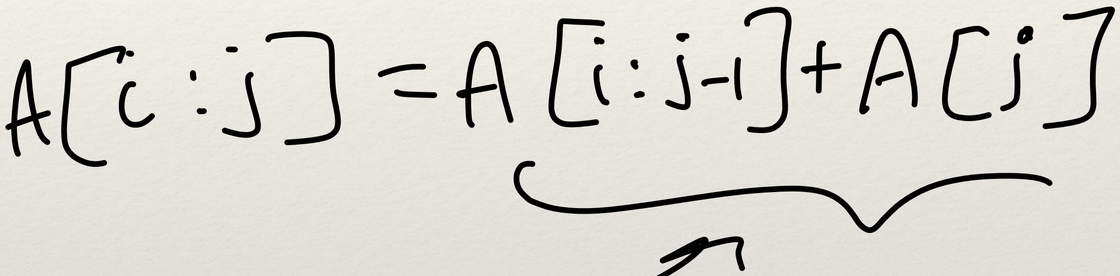


- Approach #2: Modified brute force.

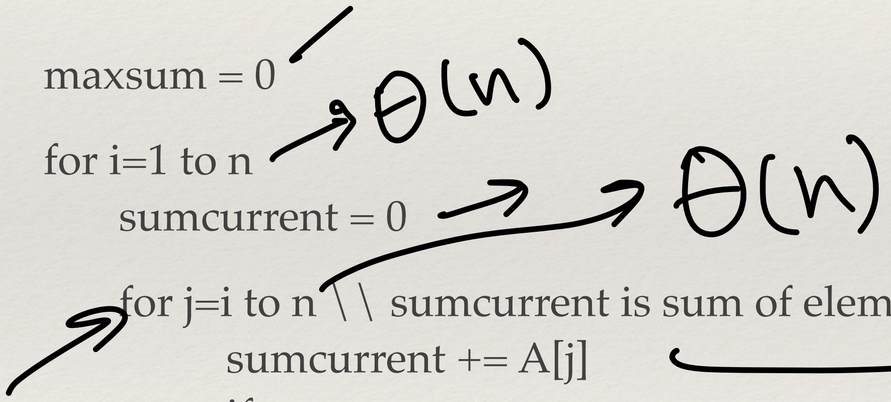
  Consider subarrays beginning in position 1, in increasing length, A[1], A[1..2], A[1…3], …, A[1…n]

  When go from A[1…j-1] to A[1…j], get sum of items in A[1…j] by adding A[j] to sum of the items in A[1…j-1]

  Pseudocode:




```
maxsum = 0
for i=1 to n
    sumcurrent = 0
    for j=i to n \\ sumcurrent is sum of elements in A[i..j-1]
        sumcurrent += A[j]
        if sumcurrent > maxsum
            maxsum = sumcurrent
return maxsum
```



- Approach #2: Modified brute force.
  Consider subarrays beginning in position 1, in increasing length, A[1], A[1..2], A[1…3], …, A[1…n]
  When go from A[1…j-1] to A[1…j], get sum of items in A[1…j] by adding A[j] to sum of the items in A[1…j-1]

  Pseudocode:

```
maxsum = 0

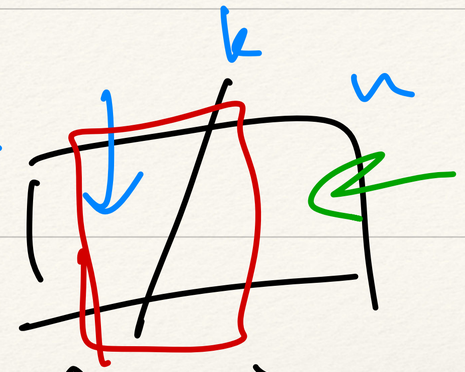
for i=1 to n
    sumcurrent = 0

    for j=i to n \\ sumcurrent is sum of elements in A[i..j-1]
        sumcurrent += A[j]
        if sumcurrent > maxsum
            maxsum = sumcurrent

return maxsum
```

$\Theta(1)$

Runtime? $\Theta(n^2) = \Theta(n) \cdot \Theta(n) \cdot \Theta(1)$

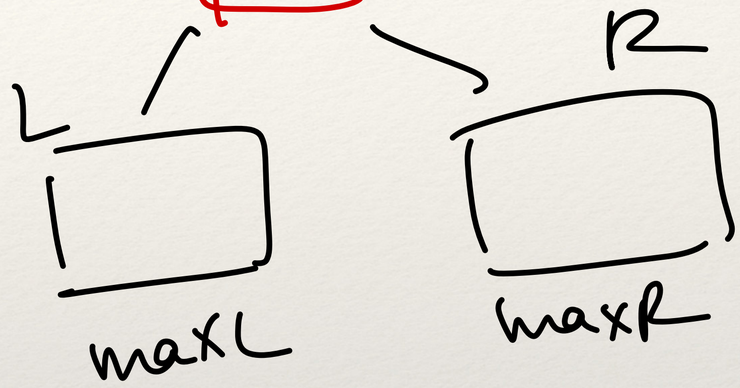


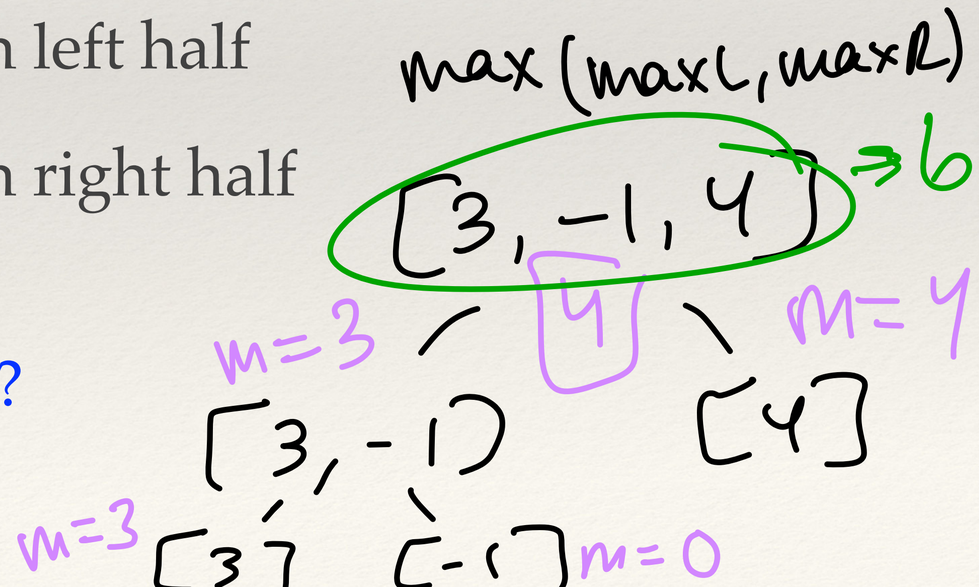
L = A[1:k] R: A[k+1:n]

- Approach 3: Divide and Conquer Approach: First Try
  - Base case

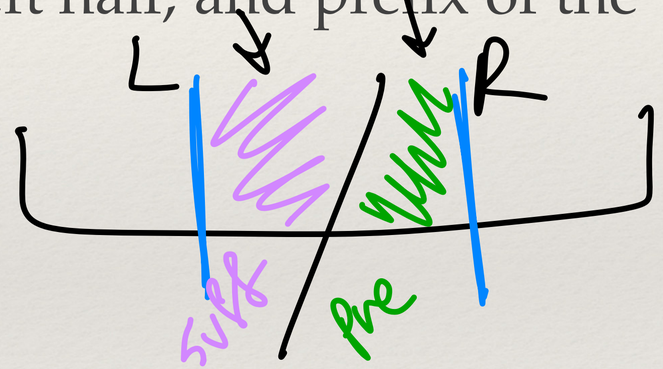
    If n=1, then if A[1] < 0 return 0 else return A[1]
  - Divide array in half
  - Recursively find maximum sum of subarray in left half
  - Recursively find maximum sum of subarray in right half
  - Compare the two sums and return the max

    What's wrong with this? How to fix?



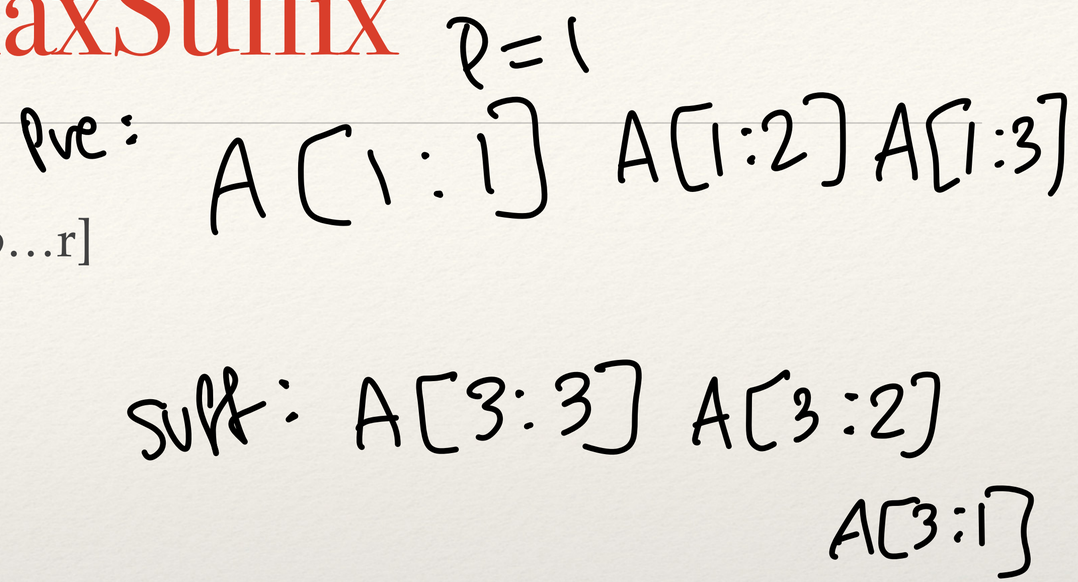


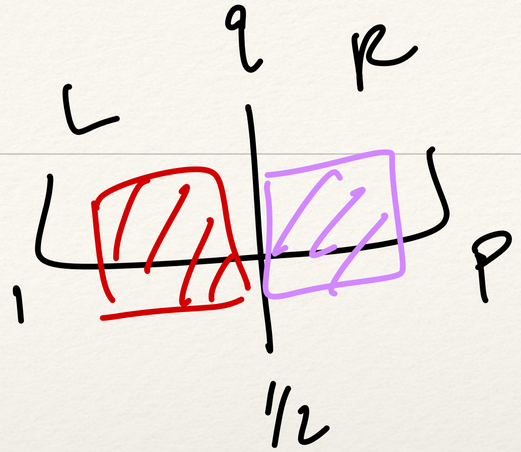
- Problem was that didn't consider subarrays that include items in left AND right subarrays (span the middle of the array)
- All arrays that span the middle consist of a suffix of the left half, and prefix of the right half
- Max sum array of this type must consist of
  - Suffix of left array having max sum
  - Prefix of right array having max sum
- Want function for finding max sum of any prefix of an array
- Want function for finding max sum of any suffix of an array



maxPrefix and maxSuffix

- maxPrefix(A,p,r) //find max sum of any prefix of A[p…r]
  - max = 0 // initialize max sum
  - prefixsum=0
  - for i=p to r
    - prefixsum+=A[i] //sum of elements in current prefix A[p…i]
    - if prefixsum > max
      - max = prefixsum
  - return max
- (maxSuffix is analogous)





- Approach 3: Divide and Conquer Approach
- MaxSum(A,p,r) // finds max sum of any subarray of A[p…r]
  - if p > r
    - return 0
  - If p == r
    - if $A[p] < 0$ return 0 else return A[p]
  - $q = \lfloor (p + r)/2 \rfloor$ // divide A[p…r] in half
  - LMax = maxSum(A,p,q) // recursively find max sum of subarray of left half
  - RMax = maxSum(q+1,r) // " " of right half
  - SuffLMax = maxSuffix(A,p,q) // find max sum of suffix on left
  - PrefRMax = maxPrefix(A,q+1,r) // find max sum of prefix on right
  - return max of LMax, RMax, (SuffLMax + PrefRMax)

- Recurrence for runtime?
  - $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$

2 recursive calls

max L max R

$q = \frac{1}{2}A$

$\Theta(n) \times 2$ $\Theta(1)$

- Runtime for our divide and conquer algorithm MaxSum
  - $T(n) = 2T(n/2) + \theta(n)$ $\nearrow n^d = n^1$
    - (same as for Mergesort)
    - $\Rightarrow T(n) = \theta(n \log n)$

$a = 2$

$b = 2$

$d = 1$

$\theta(n^d \log n)$

$\log_2 2 = 1 = d = 1$